Jethro

PROGRAMA BASES DO NEGÓCIO

Documento de revisão do fluxo de funcionamento do aplicativo — arquitetura, telas, integrações e estado atual de cada funcionalidade. Preparado para revisão com Product Owner.

10	14	24	3
TELAS	ENDPOINTS	SEMANAS DE PLANO	IAS INTEGRADAS

1 Jornada Completa do Usuário

Fluxo sequencial desde o cadastro até a execução do plano semanal

Fluxo Principal (sequência obrigatória)

ENTRADA	ACESSO	LIBERAÇÃO	CONTEXTO		EXECUÇÃO
Diagnóstico	**Autenticação**	**Paywall**	**Onboarding**	**Gerar Plano**	**App Principal**
Formulário 18 perguntas	Email / Google	Stripe checkout	~30 perguntas filtradas	Claude · 24 semanas	4 abas ativas

Gate de fluxo: O backend verifica em `GET /user/flow-status` se o usuário completou cada etapa. Qualquer etapa pendente redireciona automaticamente — o usuário não pode pular para o app sem ter plano gerado.

Gate Semanal — Lógica de Progressão

Após entrar no app, o progresso é controlado por um sistema de gates semanais:

🔒 **Locked (padrão)**
Semanas 2–24 começam bloqueadas. Conteúdo visível na Biblioteca mas não interativo.

▶ **Available (semana ativa)**
Semana 1 começa disponível. Usuário faz check-in diário (máx. 1 por dia).

✓ **Avanço de Semana**
Após 5 check-ins, botão "Avançar" fica disponível → `POST /gate/advance` → próxima semana desbloqueia.

Regras de Navegação

CONDIÇÃO	DESTINO	ENDPOINT VERIFICADO
Não autenticado	Tela de Boas-Vindas	Supabase Auth
Sem diagnóstico	Formulário de Diagnóstico	`hasDiagnostic = false`

CONDIÇÃO	DESTINO	ENDPOINT VERIFICADO
Sem assinatura ativa	Paywall	`hasSubscription = false`
Sem onboarding completo	Onboarding	`hasOnboarding = false`
Sem plano gerado	Tela de Resultado do Onboarding	`hasPlan = false`
Tudo completo	App Principal (4 abas)	Todos os gates = true

2 Telas de Entrada e Onboarding

Fluxo completo antes de o usuário acessar o app principal

Splash / Launch

✓ FUNCIONANDO

`GET /user/flow-status`

Tela de carregamento com barra de progresso animada. Determina automaticamente para onde navegar com base no estado do usuário.

- Verifica autenticação via Supabase
- Consulta 4 flags de progresso no backend
- Redireciona para a próxima etapa pendente

Boas-Vindas

✓ FUNCIONANDO

`Estático`

Landing com proposta de valor do Jethro. Apresenta os 3 pilares: Diagnóstico, Plano e Mentor. CTAs para cadastro e login.

Autenticação

✓ FUNCIONANDO

`Supabase Auth`

Login e cadastro em toggle. Suporta email + senha e OAuth Google. Recuperação de senha via email.

- Email + senha
- Google OAuth
- Reset de senha
- Link de identidades (Setup Screen)

Diagnóstico

✓ FUNCIONANDO

`POST /forms/diagnostico-inicial/submissions`

Formulário de 18 perguntas com lógica condicional. Identifica o Modelo de Diagnóstico do usuário (A–H, X).

- Lógica condicional entre perguntas
- Auto-save de rascunho local
- 8 modelos diagnósticos possíveis
- Pontuação automática no backend

Paywall

✓ FUNCIONANDO

`POST /subscription/create-checkout`

Tela de assinatura com 5 benefícios do programa. Integração Stripe: abre checkout no browser, verifica pagamento ao voltar.

- Checkout via Stripe
- Modo sandbox para testes
- Verifica status: trial / active / grace_period

Onboarding

✓ FUNCIONANDO

`GET /onboarding/questions · POST /onboarding/submit`

Questionário multi-fase com ~30 perguntas personalizadas pelo modelo diagnóstico. Gera o JSON completo do negócio do usuário.

- Fases: Básico → Receita → Comercial → Operação → Metas
- Perguntas filtradas pelo modelo diagnóstico
- Rascunho salvo localmente
- Visibilidade condicional por resposta anterior

✦ Geração do Plano de Ação (IA)

CORE FEATURE

`POST /plano/generate`

Após o onboarding, o usuário clica em "Gerar o meu plano de ação". O backend chama o Claude Sonnet com o framework ALMA v5.14 + dados do negócio para gerar um plano estratégico de 24 semanas completamente personalizado.

- Modelo: Claude Sonnet 4.6 · Output: até 32.000 tokens
- Context: ALMA v5.14 (214K chars, prompt caching ativado)
- Output: 24 semanas · nome, objetivo, por que importa, versículo, 2–4 tarefas, indicador, resultado
- Idempotente: se plano já existe, retorna o existente
- Tempo estimado: ~4–5 minutos (timeout do app: 2 minutos)

3 App Principal — 4 Abas

Telas acessíveis após o plano ser gerado

Aba 1 — Início (Home)

Tela Início

✓ FUNCIONANDO

`GET /home · POST /check-in · POST /gate/advance`

Hub central do programa. Reúne em uma tela: saudação personalizada por horário, devocional da semana, KPIs do negócio, tarefas da semana atual e controle de avanço de gate.

- ✓ **Saudação com horário** — Bom dia / tarde / noite com nome do usuário
- ✓ **Modelo de Diagnóstico** — Badge do modelo identificado (ex: Modelo D)
- ✓ **Devocional da Semana** — Título, versículo e texto da semana atual
- ✓ **KPIs do Negócio** — Receita atual, ticket médio, clientes ativos (dados do onboarding)
- ✓ **Plano da Semana** — Número da semana, objetivo, lista de tarefas com prioridade e status de conclusão
- ✓ **Gate de Avanço** — Contador X/5 check-ins, barra de progresso, botão diário e botão de avanço (quando desbloqueado)
- ✓ **Modal de Check-in** — Captura resposta do usuário sobre o trabalho do dia (cumpriu: sim/não, nota)

Aba 2 — Mentor (IA)

Chat com Mentor Jethro

✓ FUNCIONANDO

`POST /mentor/chat`

Interface de chat com o mentor IA Jethro, contextualizado no método PBN. Sugestões de perguntas pré-definidas para guiar a conversa.

- ✓ Histórico de mensagens na sessão
- ✓ Seleção de sugestões de perguntas frequentes

- ✓ Mensagem de boas-vindas automática
- ✓ Session ID gerado por conversa

Aba 3 — Biblioteca

Plano Completo de 24 Semanas

✓ FUNCIONANDO

`GET /plano`

Visualização completa do plano gerado. 24 semanas organizadas em 5 blocos estratégicos, com detalhe expandível por semana e controle de gates.

- ✓ 5 blocos colapsáveis (Fundamento · Estrutura · Controlo · Crescimento · Legado)
- ✓ Ícone de gate por semana: 🔒 locked · ▶ available · ✓ completed
- ✓ Detalhe expandível: objetivo, por que importa, versículo, tarefas com prioridade, indicador, resultado
- ✓ Cores de prioridade: crítica (vermelho) · alta (ouro) · média (azul) · baixa (cinza)
- ✓ Medidor de progresso: X/24 semanas concluídas

Aba 4 — Perfil

Tela de Perfil

✓ FUNCIONANDO

`Dados locais / sessão`

Painel do usuário com dados da conta, progresso no programa e acesso ao FAQ e ao diagnóstico.

- ✓ Avatar + email + data de membro desde
- ✓ Status da assinatura
- ✓ Grid de progresso: semana atual / 24 semanas / gates concluídos
- ✓ Barra de progresso do programa (%)
- ✓ Link para FAQ (estático)
- ✓ Link para refazer diagnóstico

- ✓ Botão de logout

4 Arquitetura Técnica

Stack completa e integrações externas do sistema

MOBILE APP
Expo · React Native
10 telas
Supabase Auth
Stripe WebView
API URL: Render

→

BACKEND API
Node.js · Fastify · TypeScript
14 endpoints
Deploy: Render
Auth: Supabase JWT
jethro-fullstack.onrender.com

→

BANCO DE DADOS
PostgreSQL · Supabase
14 tabelas
Auth integrado
20 migrações
sglpwjmdrvapurllisdn

Integrações Externas

SERVIÇO	USO	ONDE É CHAMADO	STATUS
Anthropic Claude Sonnet 4.6	Geração do Plano de 24 Semanas + Chat do Mentor	Backend: `/plano/generate`, `/mentor/chat`	✓ OK
Supabase Auth	Autenticação (email/senha, Google OAuth, JWT)	App mobile (direto) + backend (validação)	✓ OK
Stripe	Checkout de assinatura, webhook de pagamento	Backend: `/subscription/*`	✓ OK

Principais Tabelas do Banco

TABELA	FUNÇÃO	RELACIONAMENTO
users	Dados do usuário, auth_id, status	Hub central — FK em todas
diagnostico_respostas	Respostas do formulário + modelo identificado	→ onboarding_sessions

TABELA	FUNÇÃO	RELACIONAMENTO
onboarding_sessions	JSON completo do negócio (92 campos)	→ planos_acao
planos_acao	JSON do plano gerado pela IA (documento_1)	→ semanas
semanas	24 semanas com fase, pilar, objetivo	→ tarefas_semana, gates_semanais
tarefas_semana	Tarefas por semana (descrição, prioridade)	→ semanas
gates_semanais	Status de desbloqueio por semana por usuário	→ users, semanas
assinaturas	Status de assinatura Stripe (trial/active/etc.)	→ users
devocionais	Devocional por semana (1–24)	Independente

5 Mapa Completo de Endpoints

Todos os endpoints da API e seu estado atual

MÉTODO	ENDPOINT	FUNÇÃO	AUTH	STATUS
GET	/health/check	Verifica se a API está rodando	—	✓
GET	/user/flow-status	Flags de progresso do usuário no fluxo	JWT	✓
GET	/home	Dados da Home: devocional, KPIs, semana atual	JWT	✓
GET	/forms/:slug	Formulário diagnóstico (estrutura + perguntas)	—	✓
POST	/forms/:slug/submissions	Submete respostas do diagnóstico	—	✓
GET	/onboarding/questions	Perguntas filtradas por modelo diagnóstico	JWT	✓
POST	/onboarding/submit	Salva respostas do onboarding, retorna JSON do negócio	JWT	✓
GET	/onboarding/summary	Dados do onboarding para tela de resultado	JWT	✓
POST	/plano/generate	Gera plano de 24 semanas com Claude (idempotente)	JWT	✓ CORRIGIDO
GET	/plano	Plano completo: 24 semanas + tarefas + gates	JWT	✓
POST	/check-in	Registra check-in diário (máx. 1/dia, idempotente)	JWT	✓
POST	/gate/advance	Avança para próxima semana (requer 5+	JWT	✓

MÉTODO	ENDPOINT	FUNÇÃO	AUTH	STATUS
		check-ins)		
`POST`	`/mentor/chat`	Envia mensagem ao Mentor IA (Claude)	JWT	✓
`POST`	`/subscription/create-checkout`	Cria sessão Stripe e retorna URL de checkout	JWT	✓

Autenticação: Todos os endpoints com JWT validam o token Bearer via Supabase Auth. O backend cria o usuário automaticamente na tabela `users` se for o primeiro acesso autenticado.

6 Bugs Identificados e Corrigidos

Análise e resolução dos problemas encontrados no fluxo de geração do plano

Foram identificados 3 bugs na geração do plano. Todos foram corrigidos nesta sessão. Os bugs foram descobertos executando uma simulação end-to-end idêntica ao código de produção.

Bug #1 — ALMA não copiado no build (causa raiz do "Erro interno do servidor.")

CORRIGIDO

CAUSA

O comando de build (`tsc`) só compila arquivos `.ts`. O arquivo `alma-content.txt` (214KB, framework estratégico) não era copiado para `dist/lib/`. O servidor de produção no Render roda `node dist/server.js` — ao tentar ler o ALMA, lançava erro `ENOENT` (arquivo não encontrado) antes mesmo de chamar a IA. Como não havia `try-catch`, o erro caía no handler global → "Erro interno do servidor."

CORREÇÃO

package.json — script build:

`tsc && cp src/lib/alma-content.txt dist/lib/alma-content.txt`

service.ts: Envolveu `loadAlmaContent()` em try-catch, lançando `ALMA_LOAD_ERROR` com mensagem descritiva.

Bug #2 — MAX_TOKENS insuficiente para 24 semanas completas

CORRIGIDO

CAUSA

O limite de output era `MAX_TOKENS=16.000`. A simulação de produção confirmou: o modelo gera exatamente 16.000 tokens e é cortado no meio da semana 24 (JSON sem fechar). O `JSON.parse()` falha com "Unterminated string". Este seria o segundo erro revelado após o Bug #1 ser corrigido.

CORREÇÃO

Aumentado `MAX_TOKENS` de **16.000** para **32.000**.

Dados da simulação: input=1.824 tokens (user msg) + ~53K (ALMA cached) · output=16.000 tokens cortados em 50.720 chars · plano incompleto na semana 24.

Bug #3 — FK violation: acao_codigo gerado pela IA com valor inválido

CORRIGIDO

CAUSA

CORREÇÃO

O prompt pede à IA para retornar `"acao_codigo": null`, mas o modelo às vezes retorna valores inventados (ex: `"LID-07"`). Como `acoes_library` está vazia em produção, o `INSERT INTO tarefas_semana` falharia com violação de chave estrangeira dentro da transação de persistência.

No INSERT de tarefas, o valor de `acao_codigo` é agora sempre forçado para `null`, independente do que a IA retornar.

Erro sem tratamento (Anthropic SDK)

Chamada à API Anthropic sem try-catch

CORRIGIDO

CAUSA

A chamada `client.messages.create()` não estava em `try-catch`. Qualquer erro de rede, autenticação ou limite da API gerava um erro bruto (não AppError) que caía no handler genérico, mascarando a causa real.

CORREÇÃO

Chamada envolvida em try-catch. Lança `AppError` com código `AI_CALL_ERROR` e mensagem descritiva incluindo o status HTTP e erro real da API.

7 Status Geral e Próximos Passos

Visão executiva para o Product Owner

Resumo de Funcionalidades

ÁREA	FUNCIONALIDADE	TELA	STATUS
Entrada	Diagnóstico (18 perguntas, 8 modelos)	Diagnóstico	✓ OK
Acesso	Login / Cadastro / Google / Reset de senha	Auth	✓ OK
Monetização	Paywall + Stripe checkout + verificação	Paywall	✓ OK
Contexto	Onboarding 92 campos, perguntas adaptativas	Onboarding	✓ OK
Core IA	Geração do Plano 24 semanas com Claude	Resultado Onboarding	✓ CORRIGIDO
Home	KPIs, Devocional, Tarefas da semana, Gate	Início	✓ OK
IA Chat	Mentor Jethro — chat contextualizado no PBN	Mentor	✓ OK
Conteúdo	Biblioteca com 24 semanas + tarefas + gates	Biblioteca	✓ OK
Conta	Perfil, progresso, FAQ, logout	Perfil	✓ OK
Progressão	Check-in diário + Gate de avanço semanal	Início	✓ OK

Ação Necessária — Deploy

Para ativar as correções em produção: É necessário fazer um novo deploy no Render. O primeiro `npm run build` irá executar automaticamente o `cp alma-content.txt` e a geração de planos passará a funcionar. Os 3 bugs são corrigidos com este único deploy.

Observações para o PO

- ⚠ **Tempo de geração do plano (~4–5 min):** O timeout atual do app é 2 minutos. Após o deploy, testar se o plano completo é gerado dentro desse tempo ou se é necessário aumentar o timeout ou usar streaming.
- ⚠ **Status da assinatura no Perfil:** Atualmente exibido como "Activa" de forma estática. Pode ser dinamizado consultando `GET /subscription/status`.
- ✓ **Idempotência:** O plano é gerado uma única vez por usuário. Chamadas subsequentes ao `POST /plano/generate` retornam o plano existente sem custo de IA adicional.
- ✓ **Prompt Caching:** O conteúdo ALMA (214KB) é enviado com `cache_control: ephemeral`. A partir da segunda chamada, a leitura do cache reduz custo e latência em ~85%.